МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ) Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Проектирование баз данных»

Практическая работа № 1-7

Москва 2026

1.1. Бизнес-правила систем

Сложные проверки логики, обеспечивающие корректное функционирование предприятия:

1. **Контроль остатка на счёте** – при попытке создать транзакцию типа 'expense' сумма не должна превышать текущий баланс. При превышении операция блокируется..

2. **Обновление баланса счёта** – при создании, изменении или удалении транзакции автоматически пересчитывается current_balance соответствующего счёта (суммирование всех доходов и расходов по счёту

3. **Обновление накоплений цели** – при создании транзакции, привязанной к goal_id, если type='income', сумма добавляется к current_amount цели; если type='expense' – вычитается (если цель накопления, то траты с неё не предусмотрены, но может быть). При достижении target_amount статус цели меняется на 'achieved.

4. **Контроль бюджета** – при создании расходной транзакции система проверяет наличие бюджета на данную категорию в текущем месяце. Если сумма трат (spent_amount + новая транзакция) превышает лимит, пользователю выдаётся предупреждение.

5. **Единственность активной подписки** – у пользователя не может быть более одной активной подписки одновременно. При попытке создать новую подписку со статусом 'active', все предыдущие активные подписки этого пользователя автоматически завершаются (end_date = сегодня, статус = 'expired').

6. **Автоматическое создание транзакций из регулярных платежей** –ежедневно по расписанию для всех активных recurring_transaction с next_execution <= сегодня создаётся транзакция с параметрами шаблона, после чего next_execution обновляется на следующий период.

7. **Связь платежа и подписки** – при создании платежа со статусом 'success' для подписки, если подписка была неактивна, она автоматически акти-

вируется (или продлевается) в зависимости от логики.

8. **Аудит изменений** – все операции CREATE, UPDATE, DELETE над

ключевыми сущностями (пользователь, счёт, транзакция, подписка, тариф) должны автоматически записываться в audit_log с указанием пользователя, времени и изменённых данных

9. **Запрет удаления пользователя с ненулевыми счетами** – пользователь не может быть удалён (помечен как удалённый), пока у него есть счета с current_balance > 0. Необходимо сначала обнулить или закрыть счета.

10. **Каскадное обновление регулярных платежей** – при изменении счёта или категории, связанных с активным регулярным платежом, система должна проверять, что новые значения допустимы (счёт не закрыт, категория существует), иначе деактивировать шаблон

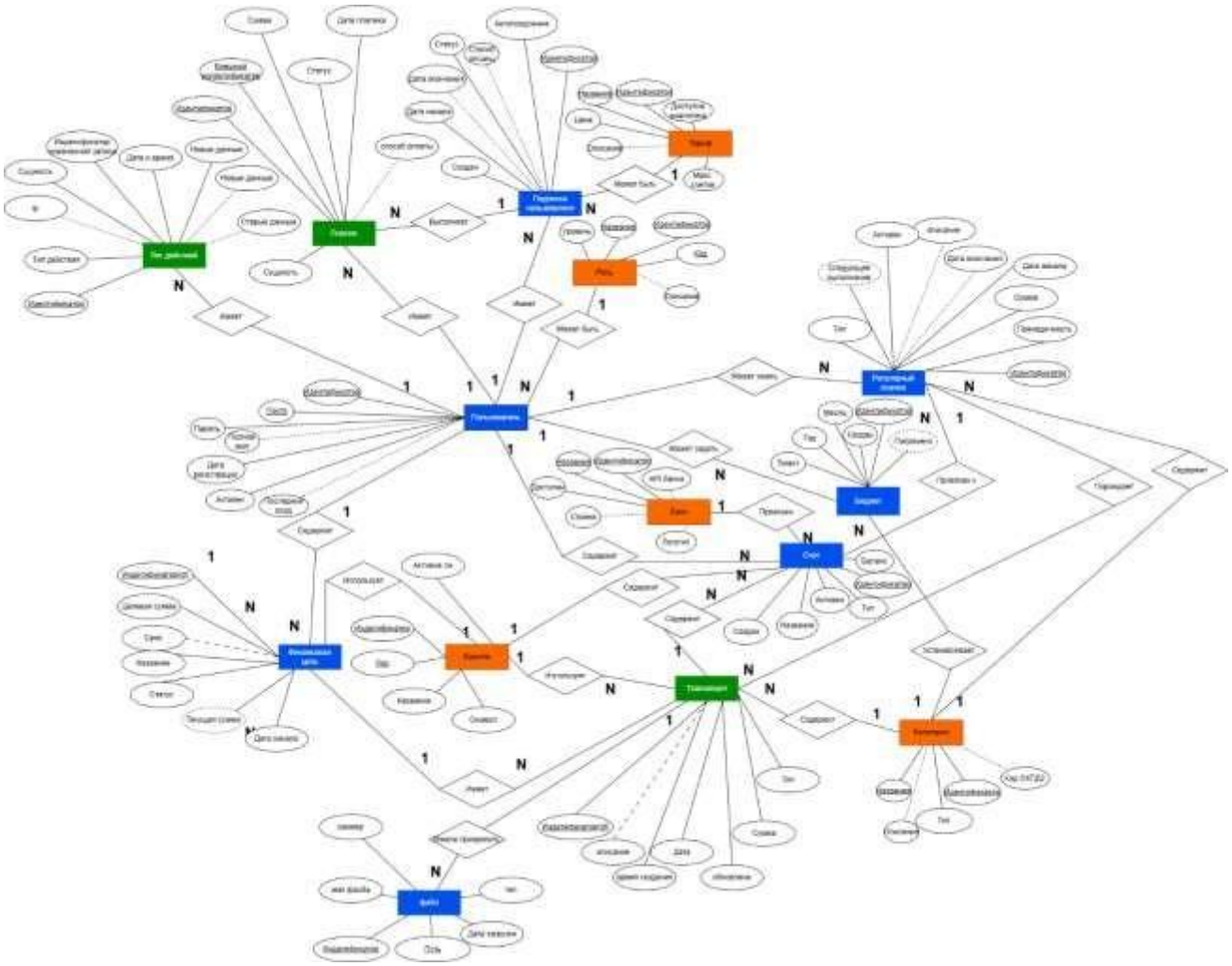

Рисунок 1- ER-диаграмма

Практическая работа № 4

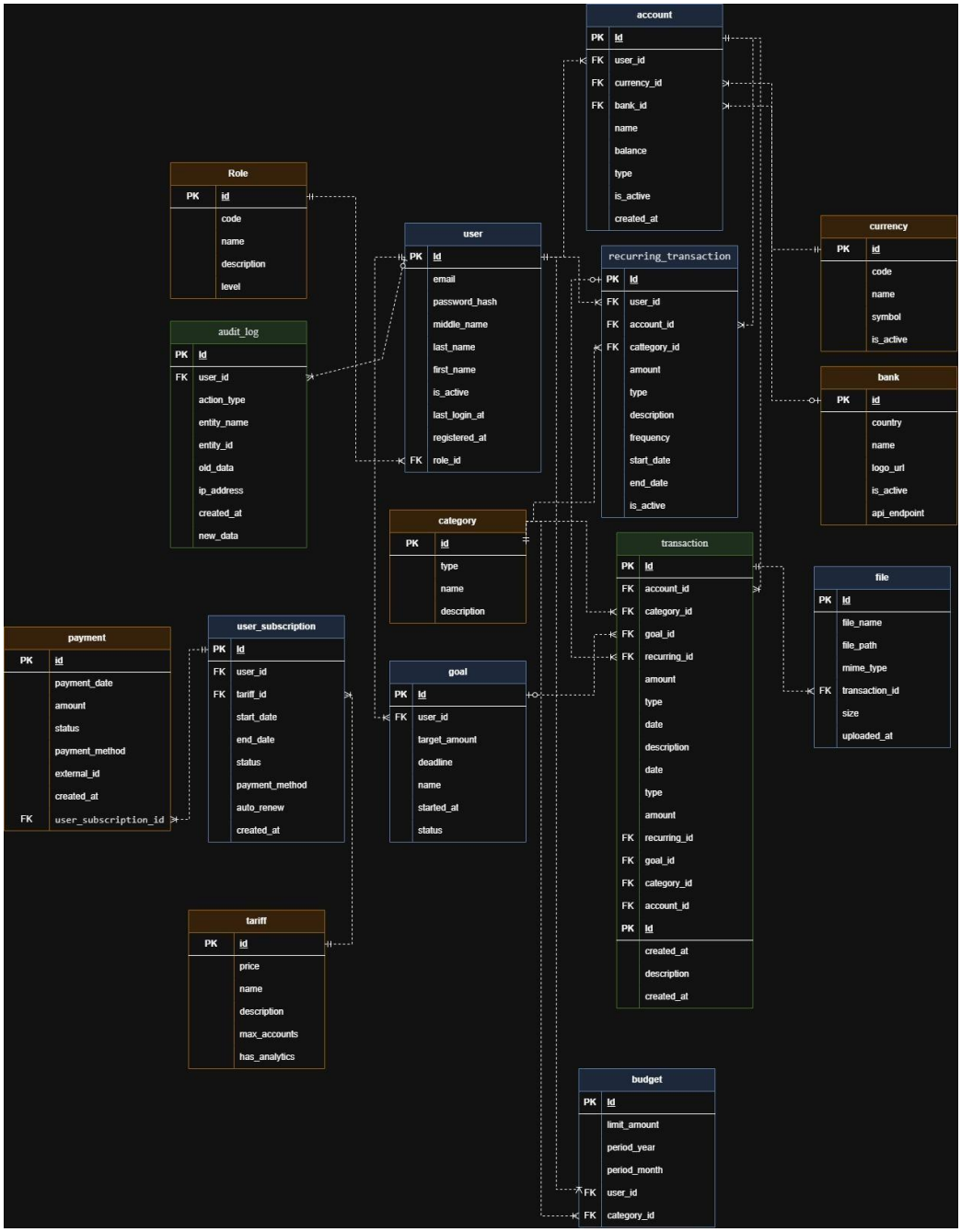

Рисунок 1 - схема

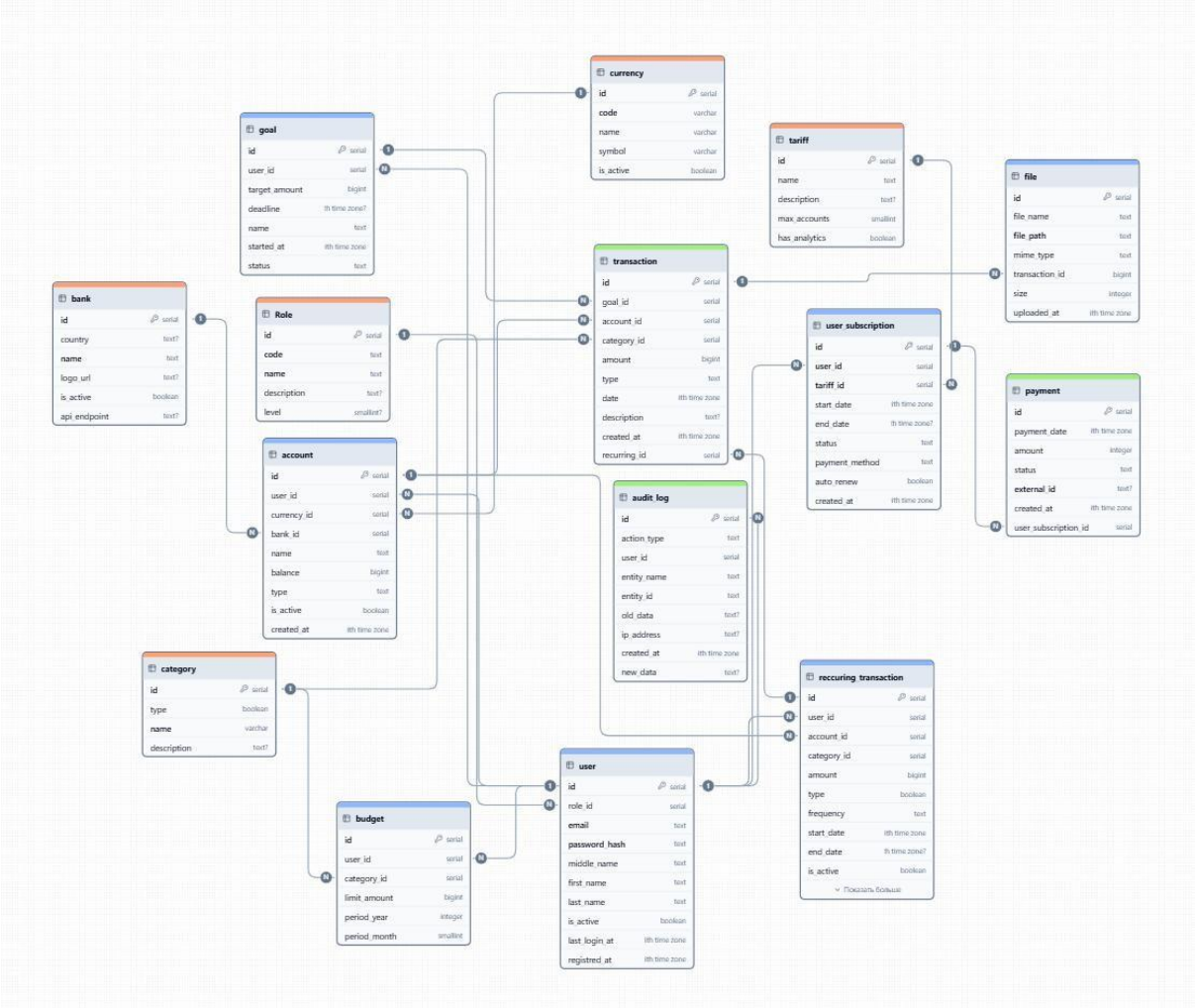

Рисунок 1- chartdb диаграмма

DDL Sql скрипт для создания таблиц

```
DROP TABLE IF EXISTS payment CASCADE;
DROP TABLE IF EXISTS transaction CASCADE;
DROP TABLE IF EXISTS budget CASCADE;
DROP TABLE IF EXISTS goal CASCADE;
DROP TABLE IF EXISTS user_subscription CASCADE;
DROP TABLE IF EXISTS recurring_transaction CASCADE;
DROP TABLE IF EXISTS account CASCADE;
DROP TABLE IF EXISTS "user" CASCADE;
DROP TABLE IF EXISTS category CASCADE;
DROP TABLE IF EXISTS tariff CASCADE;
DROP TABLE IF EXISTS bank CASCADE;
DROP TABLE IF EXISTS currency CASCADE;
DROP TABLE IF EXISTS role CASCADE;


DROP DOMAIN IF EXISTS full_name_valid CASCADE;
DROP TYPE IF EXISTS full_name_type CASCADE;


CREATE TYPE full_name_type AS (
    last_name   text,
```

```sql
    first_name  text,
    middle_name text
);

CREATE DOMAIN full_name_valid AS full_name_type CHECK (
    -- Обязательность и минимальная длина (из табл.23)
    (VALUE).last_name IS NOT NULL
    AND (VALUE).first_name IS NOT NULL
    AND length(trim((VALUE).last_name)) >= 2
    AND length(trim((VALUE).first_name)) >= 2
    AND (
        (VALUE).middle_name IS NULL
        OR length(trim((VALUE).middle_name)) >= 2
    )
    -- Максимальная длина
    AND length((VALUE).last_name) <= 50
    AND length((VALUE).first_name) <= 50
    -- Только буквы, дефис, апостроф
    AND (VALUE).last_name ~ '^[a-zA-Za-яА-ЯёЁ\-'']+$'
    AND (VALUE).first_name ~ '^[a-zA-Za-яА-ЯёЁ\-'']+$'
);
CREATE DOMAIN all_name AS varchar(50) CHECK trim(length(VALUE)>=2);

CREATE TABLE category (
    id          int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    type        boolean NOT NULL,  -- true=доход, false=расход
    name        all_name NOT NULL UNIQUE,
    description text,
    parent_id   int,
    is_system   boolean NOT NULL DEFAULT false,

    CONSTRAINT chk_category_parent_self CHECK (parent_id != id)  -- нельзя
ссылаться на себя
);

-- Таблица: Валюта (из табл.19)
CREATE TABLE currency (
    id          int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code        varchar(3) NOT NULL UNIQUE,
    name        all_name NOT NULL,
    symbol      varchar(1) NOT NULL,
    is_active   boolean NOT NULL DEFAULT true,


    CONSTRAINT chk_currency_code CHECK (code ~ '^[A-Z]{3}$'),  -- 3 заглавные буквы
(ISO)
    CONSTRAINT chk_currency_symbol CHECK (length(symbol) <= 3)
);

-- Таблица: Банк (из табл.20)
CREATE TABLE bank (
```

```
    id            int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name          all_name NOT NULL UNIQUE,
    country       text,
    api_endpoint text,
    logo_url      text,
    is_active     boolean NOT NULL DEFAULT true,
    created_at    timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_bank_api_url CHECK (api_endpoint IS NULL OR api_endpoint ~
'^https?://'),
    CONSTRAINT chk_bank_logo_url CHECK (logo_url IS NULL OR logo_url ~
'^https?://'),
    CONSTRAINT chk_bank_created_at CHECK (created_at <= CURRENT_TIMESTAMP)
);


CREATE TABLE role (
    id           int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code         text NOT NULL UNIQUE,
    name         all_name NOT NULL UNIQUE,
    description text,
    level        smallint DEFAULT 1,


    CONSTRAINT chk_role_code CHECK (code ~ '^[a-z_]+$'),  -- только латиница и
нижнее подчеркивание
    CONSTRAINT chk_role_level CHECK (level >= 0)
);

CREATE TABLE tariff (
    id              int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name            all_name NOT NULL UNIQUE,
    description     text,
    max_accounts    smallint,
    has_analytics   boolean,


    CONSTRAINT chk_tariff_max_accounts CHECK (max_accounts IS NULL OR max_accounts
> 0)
);


CREATE TABLE "user" (
    id             int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    role_id        integer NOT NULL,
    email          text NOT NULL UNIQUE,
    password_hash text NOT NULL UNIQUE,
    full_name      full_name_valid NOT NULL,
    is_active      boolean NOT NULL DEFAULT true,
    last_login_at timestamp with time time NOT NULL DEFAULT CURRENT_TIMESTAMP,
    registred_at   timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```sql
    CONSTRAINT chk_user_email CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-
]+\.[A-Za-z]{2,}$'),
    CONSTRAINT chk_user_password_hash CHECK (length(password_hash) >= 60),
    CONSTRAINT chk_user_registred_at CHECK (registred_at <= CURRENT_TIMESTAMP),
    CONSTRAINT chk_user_last_login_at CHECK (last_login_at <= CURRENT_TIMESTAMP),
    CONSTRAINT chk_user_dates CHECK (last_login_at >= registred_at)  -- последний
вход не раньше регистрации
);


CREATE TABLE account (
    id          int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id     integer NOT NULL,
    currency_id integer NOT NULL,
    bank_id     integer NOT NULL,
    name        all_name NOT NULL,
    balance     bigint NOT NULL,
    type        text NOT NULL,
    is_active   boolean NOT NULL DEFAULT true,
    created_at  timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,



    CONSTRAINT chk_account_type CHECK (type IN ('cash', 'card', 'saving')),
    CONSTRAINT chk_account_balance CHECK (balance >= 0),
    CONSTRAINT chk_account_created_at CHECK (created_at <= CURRENT_TIMESTAMP)
);


CREATE TABLE goal (
    id              int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id         integer NOT NULL,
    target_amount   bigint NOT NULL,
    current_amount  bigint NOT NULL DEFAULT 0,
    deadline        timestamp with time zone,
    name            all_name NOT NULL,
    started_at      timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status          text NOT NULL,


    CONSTRAINT chk_goal_target_amount CHECK (target_amount > 0),
    CONSTRAINT chk_goal_current_amount CHECK (current_amount >= 0),
    CONSTRAINT chk_goal_current_not_exceed_target CHECK (current_amount <=
target_amount),
    CONSTRAINT chk_goal_status CHECK (status IN ('in_progress', 'achieved',
'cancelled')),
    CONSTRAINT chk_goal_deadline CHECK (deadline IS NULL OR deadline >=
started_at),
    CONSTRAINT chk_goal_started_at CHECK (started_at <= CURRENT_TIMESTAMP)
```

```sql
);

CREATE TABLE user_subscription (
    id             int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id        integer NOT NULL UNIQUE,
    tariff_id      integer NOT NULL,
    start_date     timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    end_date       timestamp with time zone,
    status         text NOT NULL DEFAULT 'active',
    payment_method text,
    auto_renew     boolean NOT NULL DEFAULT true,
    created_at     timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,


    CONSTRAINT chk_subscription_status CHECK (status IN ('active', 'expired',
'cancelled')),
    CONSTRAINT chk_subscription_dates CHECK (end_date IS NULL OR end_date >=
start_date),
    CONSTRAINT chk_subscription_created_at CHECK (created_at <= CURRENT_TIMESTAMP)
);


CREATE TABLE recurring_transaction (
    id             int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id        integer NOT NULL,
    account_id     integer NOT NULL,
    category_id    integer NOT NULL,
    amount         numeric(10, 2) NOT NULL,
    type           boolean NOT NULL,  -- true=доход, false=расход
    frequency      text NOT NULL,
    start_date     timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
    end_date       timestamp with time zone,
    is_active      boolean NOT NULL DEFAULT true,
    description    text,
    next_execution timestamp with time zone,


    CONSTRAINT chk_recurring_amount CHECK (amount > 0),
    CONSTRAINT chk_recurring_frequency CHECK (frequency IN ('monthly', 'weekly',
'yearly')),
    CONSTRAINT chk_recurring_dates CHECK (end_date IS NULL OR end_date >=
start_date),
    CONSTRAINT chk_recurring_start_date CHECK (start_date >= CURRENT_DATE)  --
нельзя создать в прошлом
);


CREATE TABLE budget (
    id            int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id       integer NOT NULL,
    category_id   integer NOT NULL,
```

```sql
    limit_amount  bigint NOT NULL,
    period_year   integer NOT NULL,
    period_month  smallint NOT NULL,


    CONSTRAINT chk_budget_limit CHECK (limit_amount > 0),
    CONSTRAINT chk_budget_period_month CHECK (period_month BETWEEN 1 AND 12),
    CONSTRAINT chk_budget_period_year CHECK (period_year >= 2000),
    CONSTRAINT chk_budget_period_year_future CHECK (period_year <= EXTRACT(YEAR
FROM CURRENT_DATE) + 10),

    -- Уникальность: один бюджет на категорию в месяц
    CONSTRAINT uk_budget_user_category_month UNIQUE (user_id, category_id,
period_year, period_month)
);


CREATE TABLE transaction (
    id           int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    goal_id      integer,
    account_id   integer NOT NULL,
    category_id  integer,
    amount       numeric(10, 2) NOT NULL,
    type         boolean NOT NULL,  -- true=доход, false=расход
    description  text,
    created_at   timestamp with time time NOT NULL DEFAULT CURRENT_TIMESTAMP,
    recurring_id integer,


    CONSTRAINT chk_transaction_amount CHECK (amount > 0),
    CONSTRAINT chk_transaction_created_at CHECK (created_at <= CURRENT_TIMESTAMP)
);


CREATE TABLE payment (
    id                    int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    payment_date          timestamp with time zone NOT NULL,
    amount                numeric(10, 2) NOT NULL,
    status                text NOT NULL DEFAULT 'pending',
    external_id           text UNIQUE,
    created_at            timestamp with time zone NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    user_subscription_id  integer NOT NULL,


    CONSTRAINT chk_payment_amount CHECK (amount > 0),
    CONSTRAINT chk_payment_status CHECK (status IN ('success', 'failed', 'pending',
'refunded')),
    CONSTRAINT chk_payment_date CHECK (payment_date <= CURRENT_TIMESTAMP),
    CONSTRAINT chk_payment_created_at CHECK (created_at <= CURRENT_TIMESTAMP),
```

```sql
    CONSTRAINT chk_payment_dates CHECK (created_at >= payment_date)  -- запись не
может быть раньше платежа
);

-- Таблица: Лог действий (из табл.27)
CREATE TABLE audit_log (
    id           int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    action_type  text NOT NULL,
    user_id      integer,
    entity_name  text NOT NULL,
    entity_id    text NOT NULL,
    old_data     text,
    new_data     text,
    ip_address   text,
    created_at   timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,


    CONSTRAINT chk_audit_action_type CHECK (action_type IN ('INSERT', 'UPDATE',
'DELETE', 'LOGIN', 'LOGOUT')),
    CONSTRAINT chk_audit_ip CHECK (ip_address IS NULL OR ip_address ~
'^(\d{1,3}\.){3}\d{1,3}$|^[0-9a-f:]+$'),
    CONSTRAINT chk_audit_created_at CHECK (created_at <= CURRENT_TIMESTAMP),
    CONSTRAINT chk_audit_entity_id CHECK (entity_id ~ '^\d+$')  -- ID должен быть
числом
);

-- Таблица: Файл (добавляем ограничения)
CREATE TABLE file (
    id             int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    file_name      text NOT NULL,
    file_path      text NOT NULL UNIQUE,
    mime_type      text NOT NULL,
    transaction_id integer NOT NULL,
    size           integer NOT NULL,
    uploaded_at    timestamp with time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,


    CONSTRAINT chk_file_size CHECK (size > 0 AND size <= 10485760),  -- до 10 МБ
    CONSTRAINT chk_file_mime CHECK (mime_type IN ('image/jpeg', 'image/png',
'image/gif', 'application/pdf')),
    CONSTRAINT chk_file_uploaded_at CHECK (uploaded_at <= CURRENT_TIMESTAMP)
);

ALTER TABLE "user" ADD CONSTRAINT user_role_id_fk FOREIGN KEY (role_id) REFERENCES
role (id) ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE account ADD CONSTRAINT account_user_id_fk FOREIGN KEY (user_id)
REFERENCES "user" (id) ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE account ADD CONSTRAINT account_currency_id_fk FOREIGN KEY (currency_id)
REFERENCES currency (id) ON DELETE RESTRICT ON UPDATE RESTRICT;
```

```sql
ALTER TABLE account ADD CONSTRAINT account_bank_id_fk FOREIGN KEY (bank_id)
REFERENCES bank (id) ON DELETE RESTRICT ON UPDATE RESTRICT;

ALTER TABLE recurring_transaction ADD CONSTRAINT recurring_user_id_fk FOREIGN KEY
(user_id) REFERENCES "user" (id) ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE recurring_transaction ADD CONSTRAINT recurring_account_id_fk FOREIGN
KEY (account_id) REFERENCES account (id) ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE recurring_transaction ADD CONSTRAINT recurring_category_id_fk FOREIGN
KEY (category_id) REFERENCES category (id) ON DELETE RESTRICT ON UPDATE RESTRICT;

ALTER TABLE budget ADD CONSTRAINT budget_user_id_fk FOREIGN KEY (user_id)
REFERENCES "user" (id) ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE budget ADD CONSTRAINT budget_category_id_fk FOREIGN KEY (category_id)
REFERENCES category (id) ON DELETE RESTRICT ON UPDATE RESTRICT;

ALTER TABLE user_subscription ADD CONSTRAINT subscription_user_id_fk FOREIGN KEY
(user_id) REFERENCES "user" (id) ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE user_subscription ADD CONSTRAINT subscription_tariff_id_fk FOREIGN KEY
(tariff_id) REFERENCES tariff (id) ON DELETE RESTRICT ON UPDATE RESTRICT;

ALTER TABLE goal ADD CONSTRAINT goal_user_id_fk FOREIGN KEY (user_id) REFERENCES
"user" (id) ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE transaction ADD CONSTRAINT transaction_goal_id_fk FOREIGN KEY (goal_id)
REFERENCES goal (id) ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE transaction ADD CONSTRAINT transaction_account_id_fk FOREIGN KEY
(account_id) REFERENCES account (id) ON DELETE RESTRICT ON UPDATE RESTRICT;
ALTER TABLE transaction ADD CONSTRAINT transaction_category_id_fk FOREIGN KEY
(category_id) REFERENCES category (id) ON DELETE RESTRICT ON UPDATE RESTRICT;
ALTER TABLE transaction ADD CONSTRAINT transaction_recurring_id_fk FOREIGN KEY
(recurring_id) REFERENCES recurring_transaction (id) ON DELETE SET NULL ON UPDATE
CASCADE;

ALTER TABLE file ADD CONSTRAINT file_transaction_id_fk FOREIGN KEY (transaction_id)
REFERENCES transaction (id) ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE payment ADD CONSTRAINT payment_subscription_id_fk FOREIGN KEY
(user_subscription_id) REFERENCES user_subscription (id) ON DELETE CASCADE ON
UPDATE CASCADE;

ALTER TABLE audit_log ADD CONSTRAINT audit_log_user_id_fk FOREIGN KEY (user_id)
REFERENCES "user" (id) ON DELETE SET NULL ON UPDATE CASCADE;

-- Дополнительное ограничение: категория может ссылаться на родительскую категорию
ALTER TABLE category ADD CONSTRAINT category_parent_id_fk FOREIGN KEY (parent_id)
REFERENCES category (id) ON DELETE SET NULL;
```

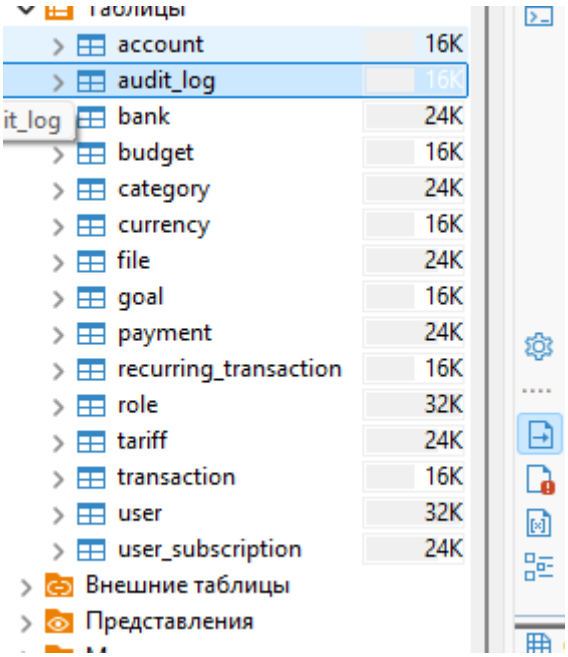

Рисунок 1- дерево навигатора

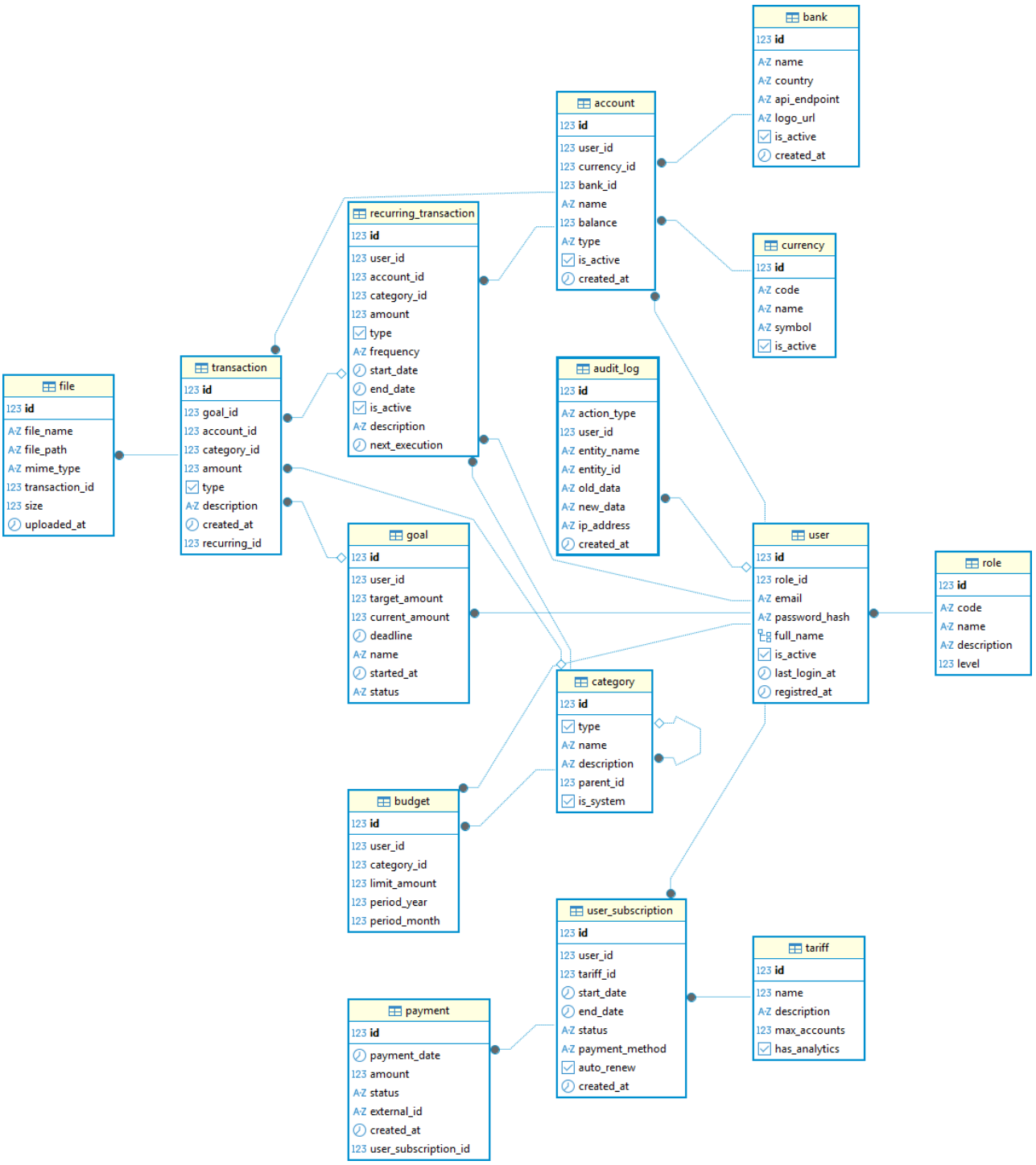

Рисунок 2- ER-диаграмма в DBeaver

Результат 1 | Статистика 1

INSERT INTO audit_log (user_id, actior | Введите SQL выражение что

	table_name	row_count
1	account	17
2	audit_log	12
3	bank	10
4	budget	17
5	category	12
6	currency	10
7	file	10
8	goal	10
9	payment	17
10	recurring_transaction	10
11	role	5
12	tariff	5
13	transaction	13
14	user	10
15	user_subscription	12

Рисунок 3- статистика заполнения табилц в DBeaver

```sql
SELECT
    id,
    email,
    (full_name).last_name,
    (full_name).first_name,
    (full_name).middle_name
FROM "user";
```

T id, email, (full_name).l | Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z email	A-Z last_name	A-Z first_name	A-Z middle_name
6	elena.volkova@out	Волкова	Елена	Михайловна
8	analyst@finance.ru	Аналитиков	Андрей	Викторович
10	viewer@finance.ru	Наблюдателей	Павел	Романович
1	admin@finance.ru	Администраторов	Иван	Петрович
2	ivan.petrov@mail.r	Петров	Иван	Алексеевич
3	anna.smirnova@gr	Смирнова	Анна	Сергеевна
4	dmitry.kozlov@yan	Козлов	Дмитрий	Николаевич
5	support@finance.r	Техническая	Поддержка	Системная
7	sergey.novikov@gr	Новиков	Сергей	Андреевич
9	mariia.sokolova@n	Соколова	Мария	Дмитриевна

Рисунок 4- select запрос в DBeaver